

# Motorsteuerung

## Dokumentation

Autor: Lucas Tavares Amaral Silva  
Studiengang: ESE  
Matrikelnummer: 225588  
Hardware: STK500-Board, ATmega 16

## 1. Aufgabenstellung und Anforderungen

---

Auf dem STK500-Board (Mikrocontroller ATmega16) soll die Ansteuerung eines Motors simuliert werden. Die Motorsteuerung wird durch LEDs und Tasten emuliert. Folgende funktionale Anforderungen sind zu erfüllen:

| Anforderung        | Details                                             |
|--------------------|-----------------------------------------------------|
| Startzustand       | LED1 (PB1) leuchtet – Motor ist aus                 |
| Motor Anlauf (S1)  | LED1 aus, LED3 (PB3) blinkt mit 1 Hz (500ms an/aus) |
| Frequenzerhöhung   | Jede Sekunde wird die Frequenz um 1 Hz erhöht       |
| Nenndrehzahl       | Nach 10 Sekunden leuchtet LED3 dauerhaft            |
| Motor Bremsen (S2) | LED3 blinkt mit 10 Hz (50ms an/aus)                 |
| Frequenzverringern | Jede Sekunde wird die Frequenz um 1 Hz verringert   |
| Motor AUS          | Nach 10 Sekunden Bremszeit leuchtet LED1 wieder     |
| Kein Blocking      | Keine delay()-Funktionen erlaubt                    |

## 2. Architektur und Lösungsansatz

---

### 2.1 Projektstruktur

Das Projekt ist nach dem Prinzip der Trennung von Zuständigkeiten in mehrere Dateien aufgeteilt. Es gibt keine blockierenden Warteschleifen – die gesamte Zeitsteuerung erfolgt über den Hardware-Timer1 des ATmega16.

Timer1 wird verwendet, weil er ein 16-Bit-Zähler ist und damit einen Wertebereich von 0 bis 65535 abdeckt. Im Vergleich dazu haben Timer0 und Timer2 nur 8 Bit (0–255), was für Zeitspannen im Sekundenbereich nicht ausreicht. Mit Prescaler 1024 und  $F_{CPU} = 8 \text{ MHz}$  läuft Timer1 ca. 8,3 Sekunden, bevor er überläuft – lang genug für die 10-Sekunden-Anlauf- und Bremszeit.

Das Prinzip: In jedem Aufruf von `motor_run()` wird die Differenz des aktuellen TCNT1-Wertes zum letzten gespeicherten Wert berechnet. Diese Differenz (in Ticks) wird auf interne Timer-Variablen addiert.

Sobald ein Schwellwert überschritten wird, wird eine Aktion ausgeführt und der Timer zurückgesetzt. So blockiert der Prozessor nie, d. h., es wird nur geprüft, ob genug Zeit vergangen ist.

| Datei       | Typ      | Beschreibung                                                                    |
|-------------|----------|---------------------------------------------------------------------------------|
| src/main.c  | C-Quelle | Einstiegspunkt, ruft <code>motor_init()</code> und <code>motor_run()</code> auf |
| src/motor.c | C-Quelle | Zustandsmaschine, Timer, LED-Steuerung, NOT-AUS                                 |
| inc/motor.h | Header   | Deklaration <code>motor_init()</code> , <code>motor_run()</code>                |
| Makefile    | Build    | Kompilierung und Flash-Vorgang mit <code>avr-gcc</code> / <code>avrdude</code>  |

## 2.2 Timer-basierte Zeitsteuerung

Da keine `delay()`-Funktionen verwendet werden dürfen, wird Timer1 des ATmega16 im Normal Mode mit Prescaler 1024 betrieben.

Der Zähler TCNT1 läuft frei von 0 bis 65535. In jedem Aufruf von `motor_run()` wird die vergangene Zeit seit dem letzten Aufruf in Ticks gemessen:

```
uint16_t now = TCNT1;
uint16_t diff = now - last; // Überlauf automatisch korrekt (uint16_t)
last = now;
```

Die Subtraktion funktioniert auch beim Überlauf korrekt, da beide Variablen `uint16_t` sind – die unsigned Arithmetik behandelt den Überlauf automatisch.

Die gemessenen Ticks werden auf `step_timer` und `blink_timer` addiert. Sobald ein Schwellwert erreicht wird, wird die entsprechende Aktion ausgeführt und der Timer zurückgesetzt:

- `blink_timer >= half_period`, bedeutet: LED3 toggeln
- `step_timer >= 1000 * TICKS_PER_MS`, bedeutet: Nächster Frequenzschritt

Die Frequenz wird durch die Halbperiode gesteuert:

- **Anlauf:**  $T_{\text{halb}} = 500\text{ms} / (\text{step} + 1)$ , bedeutet: Frequenz steigt von 1 Hz auf 10 Hz
- **Bremsen:**  $T_{\text{halb}} = 500\text{ms} / (10 - \text{step})$ , bedeutet: Frequenz sinkt von 10 Hz auf 1 Hz

Statt eines Interrupts wird TCNT1 direkt abgelesen – ein ISR wäre hier mehr Aufwand als Nutzen, weil dann Variablen zwischen ISR und Hauptprogramm geteilt und mit volatile abgesichert werden müssten. So reicht ein einfaches Ablesen.

## 2.3 Frequenz-Berechnung

Die Blinkfrequenz der LED3 wird durch die Halbperiode gesteuert – also die Zeit zwischen zwei LED-Toggles.

Je kürzer die Halbperiode, desto höher die Frequenz.

**Anlauf** (**MOTOR\_STARTING**) – Die Frequenz steigt jede Sekunde um 1 Hz:

```
uint16_t half_period = (500 * TICKS_PER_MS) / (step + 1);
```

**Bremsen** (**MOTOR BRAKING**) – Die Frequenz sinkt jede Sekunde um 1 Hz:

```
uint16_t half_period = (500 * TICKS_PER_MS) / (10 - step);
```

| Sekunde   | step | Anlauf-Frequenz  | Brems-Frequenz | T_half         |
|-----------|------|------------------|----------------|----------------|
| 0 – 1 s   | 0    | 1 Hz             | 10 Hz          | 500 ms / 50 ms |
| 1 – 2 s   | 1    | 2 Hz             | 9 Hz           | 250 ms / 55 ms |
| 2 – 3 s   | 2    | 3 Hz             | 8 Hz           | 166 ms / 62 ms |
| ...       | ...  | ...              | ...            | ...            |
| 9 – 10 s  | 9    | 10 Hz            | 1 Hz           | 50 ms / 500 ms |
| nach 10 s | -    | Dauerlicht / AUS | -              | -              |

Nach Ablauf der 10 Sekunden wechselt der Zustand automatisch:

beim Anlauf zu **MOTOR\_RUNNING** (LED3 Dauerlicht), beim Bremsen zu **MOTOR\_OFF** (LED1 AN).

## 2.4 Build-System und Entwicklungsumgebung

Das Projekt wird mit einem Makefile gebaut. Ein Makefile beschreibt wie der Compiler aufgerufen wird – welche Dateien kompiliert werden, welche Flags gesetzt werden und wie der Flash-Vorgang abläuft.

Da Microchip Studio nur unter Windows verfügbar ist, wurde das Projekt auf macOS mit `avr-gcc` direkt über das Terminal entwickelt. `avr-gcc` und `avrdude` werden über Homebrew installiert:

```
brew install avr-gcc avrdude # Installiert avr-gcc und avrdude
```

Danach kann das Projekt mit drei Befehlen vollständig bedient werden:

```
make          # Kompiliert alle .c Dateien und erzeugt die .hex Datei
make flash    # Flasht die .hex Datei auf das STK500-Board
make clean    # Löscht den build-Ordner
```

### Öffnen in Microchip Studio (Windows)

Falls das Projekt unter Windows mit Microchip Studio geöffnet werden soll, geht man folgendermaßen vor:

1. Microchip Studio öffnen
2. File → New → Project → GCC C Executable Project
3. Device ATmega16 auswählen
4. Dateien aus `src/` und `inc/` manuell einfügen (Add Existing Item)
5. Unter Project → Properties → Toolchain → AVR/GNU C Compiler → Directories den Pfad `../inc` als Include-Verzeichnis eintragen
6. Das Define `F_CPU=8000000UL` unter Preprocessor hinzufügen

Das Makefile selbst wird von Microchip Studio nicht verwendet – es generiert sein eigenes Build-System intern. Der Code bleibt jedoch vollständig kompatibel.

### 3. Ablaufplan – Zustandsübergangs- und Flussdiagramm

---

Die Motorsteuerung ist als endliche Zustandsmaschine (FSM – Finite State Machine) implementiert. Es gibt vier Zustände:

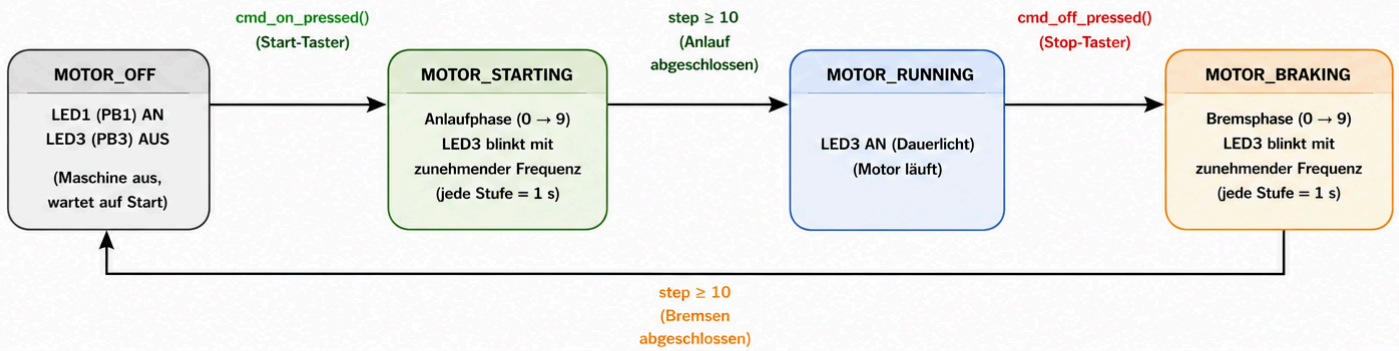
| Zustand                     | LED1 | LED3       | Beschreibung                         |
|-----------------------------|------|------------|--------------------------------------|
| <code>MOTOR_OFF</code>      | AN   | AUS        | Motor aus, wartet auf S1             |
| <code>MOTOR_STARTING</code> | AUS  | Blinkt     | Anlauf, Frequenz steigt jede Sekunde |
| <code>MOTOR_RUNNING</code>  | AUS  | Dauerlicht | Nenn Drehzahl erreicht               |
| <code>MOTOR BRAKING</code>  | AUS  | Blinkt     | Bremsen, Frequenz sinkt jede Sekunde |

Die Übergänge zwischen den Zuständen werden durch Tastenereignisse oder abgelaufene Zeitspannen ausgelöst:

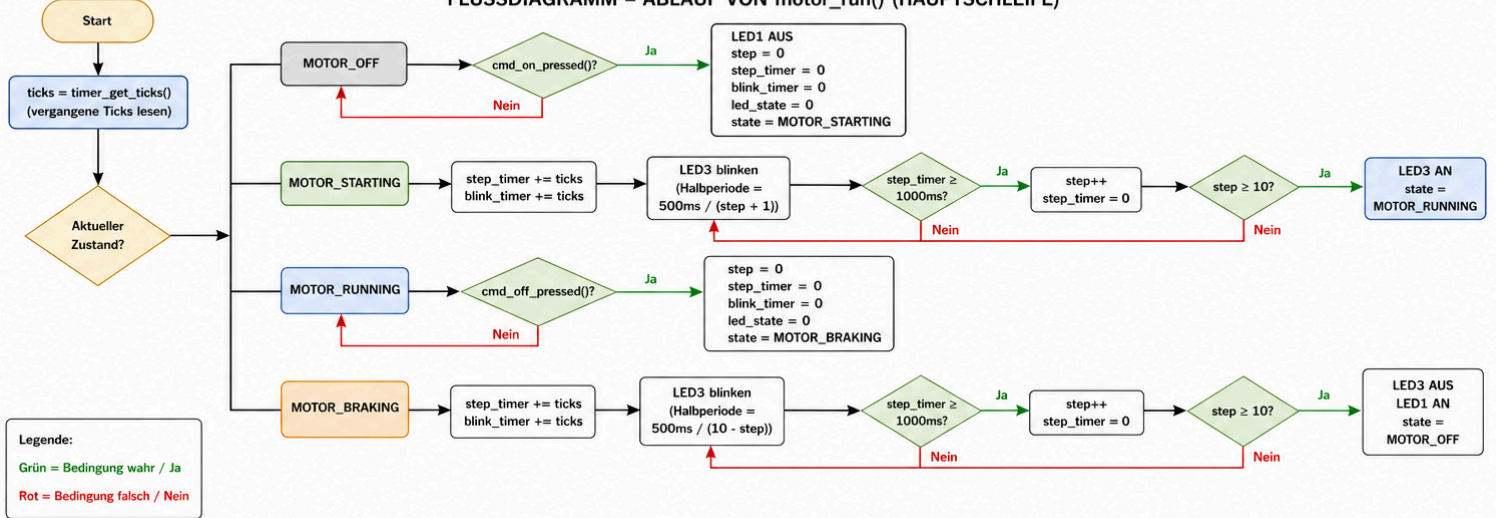
| Von                         | Nach                        | Bedingung             |
|-----------------------------|-----------------------------|-----------------------|
| <code>MOTOR_OFF</code>      | <code>MOTOR_STARTING</code> | S1 gedrückt           |
| <code>MOTOR_STARTING</code> | <code>MOTOR_RUNNING</code>  | 10 Sekunden vergangen |
| <code>MOTOR_RUNNING</code>  | <code>MOTOR BRAKING</code>  | S2 gedrückt           |
| <code>MOTOR BRAKING</code>  | <code>MOTOR_OFF</code>      | 10 Sekunden vergangen |

Die Zustandsmaschine wird in `motor_run()` durch eine `switch`-Anweisung realisiert. In jedem Durchlauf der `while(1)`-Schleife in `main.c` wird `motor_run()` einmal aufgerufen – so wird jeder Zustand kontinuierlich überprüft ohne den Prozessor zu blockieren.

## ZUSTANDSÜBERGANGSDIAGRAMM



## FLUSSDIAGRAMM – ABLAUF VON motor\_run() (HAUPTSCHLEIFE)





#### 4. Auflistung der Variablen

---

| Variable                     | Typ                  | Wertebereich                          | Bedeutung                                           |
|------------------------------|----------------------|---------------------------------------|-----------------------------------------------------|
| state                        | MotorState<br>(enum) | OFF / STARTING /<br>RUNNING / BRAKING | Aktueller Zustand der FSM                           |
| step                         | uint8_t              | 0 – 9                                 | Aktueller Anlauf- oder<br>Bremsschritt              |
| step_timer                   | uint16_t             | 0 – 8000 Ticks                        | Zeitmessung innerhalb<br>eines Schritts (1 Sekunde) |
| blink_timer                  | uint16_t             | 0 – 4000 Ticks                        | Zeitmessung für<br>LED3-Toggle                      |
| led_state                    | uint8_t              | 0 oder 1                              | Aktueller Zustand der<br>LED3 (an/aus)              |
| last (in<br>timer_get_ticks) | uint16_t<br>(static) | 0 – 65535                             | Letzter TCNT1-Wert für<br>Differenzberechnung       |

## 5. Auflistung der Funktionen

---

| Funktion                       | Input                    | Output                           | Bedeutung                                                     |
|--------------------------------|--------------------------|----------------------------------|---------------------------------------------------------------|
| <code>motor_init()</code>      | void                     | void                             | Initialisiert DDR-Register, LEDs, Pull-Ups und Timer1         |
| <code>motor_run()</code>       | void                     | void                             | Führt einen FSM-Schritt aus, ohne den Prozessor zu blockieren |
| <code>led_on(pin)</code>       | <code>uint8_t pin</code> | void                             | Setzt Pin LOW → LED leuchtet (Active LOW auf STK500)          |
| <code>led_off(pin)</code>      | <code>uint8_t pin</code> | void                             | Setzt Pin HIGH → LED aus                                      |
| <code>cmd_on_pressed()</code>  | void                     | <code>uint8_t</code><br>(0/1)    | Prüft ob S1 gedrückt ist (PIND, Active LOW)                   |
| <code>cmd_off_pressed()</code> | void                     | ( <code>uint8_t</code><br>(0/1)) | Prüft ob S2 gedrückt ist (PIND, Active LOW)                   |
| <code>timer_get_ticks()</code> | void                     | <code>uint16_t</code>            | Gibt vergangene Ticks seit letztem Aufruf zurück              |

## 6. Hardware-Konfiguration

---

### 6.1 Pin-Belegung

| Pin | Richtung | Funktion                         | Beschreibung                                    |
|-----|----------|----------------------------------|-------------------------------------------------|
| PB1 | Ausgang  | LED1 ( <b>MOTOR_STATUS_OFF</b> ) | Leuchtet wenn Motor aus (Active LOW)            |
| PB3 | Ausgang  | LED3 ( <b>MOTOR_STATUS_ON</b> )  | Leuchtet/blinkt bei Motor aktiv (Active LOW)    |
| PD1 | Eingang  | S1 ( <b>MOTOR_STATUS_ON</b> )    | Taste Motor AN (Pull-Up, Active LOW)            |
| PD2 | Eingang  | S2 ( <b>MOTOR_STATUS_OFF</b> )   | Taste Motor AUS / Bremsen (Pull-Up, Active LOW) |

## 7. Weitere Entwicklung

---

Die Tasten S1 und S2 werden im aktuellen Projekt über PORT-D eingelesen (PD1 und PD2). Beim ATmega16 liegen auf PORT-D jedoch auch die Pins der seriellen Schnittstelle USART: PD0 = RXD (Empfang) und PD1 = TXD (Senden).

Das bedeutet: Der Eingang von S1 (PD1) belegt denselben Pin wie der USART-Sendepin TXD. Solange S1 auf PD1 liegt, kann die USART-Schnittstelle nicht sauber genutzt werden, da TXD blockiert ist. Im aktuellen Projekt ist das unkritisch, weil keine serielle Kommunikation benötigt wird – für eine weitere Entwicklung (z. B. Debug-Ausgaben, Telemetrie, Logging der Drehzahlstufen oder ein serielles Kommando-Interface) wäre die USART aber sehr nützlich.

Für eine solche Erweiterung empfiehlt es sich, die beiden Tasten-Eingänge von PORT-D auf PORT-A zu verlegen (z. B. PA0 und PA1). PORT-A wird beim ATmega16 ansonsten nur vom ADC verwendet und besitzt keine Alternativfunktionen, die mit der Tasten-Abfrage kollidieren. Dadurch werden PD0 (RXD) und PD1 (TXD) vollständig für die USART frei.

Da die Tasten im aktuellen Entwurf ohnehin per Polling abgefragt werden (kein externer Interrupt), ist der Verzicht auf den Interrupt-Pin INT0 (PD2) unproblematisch – die Umstellung erfordert lediglich das Anpassen der Defines `MOTOR_CMD_*` sowie der zugehörigen Register (`PORTA`, `DDRA`, `PINA`).

|                                      | Aktuell <code>PORTD</code> | Vorschlag ( <code>PORTA</code> ) |
|--------------------------------------|----------------------------|----------------------------------|
| S1 ( <code>MOTOR_STATUS_ON</code> )  | PD1 (= TXD!)               | PA0                              |
| S2 ( <code>MOTOR_STATUS_OFF</code> ) | PD2 (= INT0)               | PA1                              |
| USART RXD/TXD                        | belegt durch S1            | frei (PD0/PD1)                   |

## 8. Quellen

---

- Projektrepository: <https://github.com/lucastavaresas-lang/Motorsteuerung>
- Atmel Corporation: *ATmega16 Datasheet*, Rev. 2466T–AVR–07/10
- Atmel Corporation: *STK500 User Guide*, Rev. 1925C–AVR–05/06
- Hochschule Heilbronn: *Laborunterlagen Mikrocontroller SS-2026*
- AVR-GCC Online-Dokumentation: <https://gcc.gnu.org/wiki/avr-gcc>
- avrdude Dokumentation: [https://avrdude.nongnu.org/avrdude\\_toc.html](https://avrdude.nongnu.org/avrdude_toc.html)
- `.gitignore` Vorlage für C-Projekte:  
<https://github.com/github/gitignore/blob/main/C.gitignore>
- Ordnerstruktur inspiriert an: <https://github.com/torvalds/linux>